

심화 실습 평가 2 (리눅스 프로그래밍)

작성자: 김예훈

프로젝트 개요

실습 주제

- TCP 소켓 통신 기반 원격 장치 제어 프로그램

개발 목표

라즈베리파이 4(서버)와 우분투 리눅스(클라이언트) 간의 안정적인 원격 TCP 통신 환경 구축

- 장치 제어 로직을 공유 라이브러리(`.so`) 형태로 모듈화 후 동적 링킹
- 백그라운드 데몬(Daemon) 프로세스 운영 및 시그널 예외 처리

시스템 아키텍처 요약

- 서버(라즈베리파이): 데몬 형식 구동, 클라이언트 접속 처리 및 독립적인 하드웨어 제어 스레드 운용
 - 클라이언트(우분투): 멀티스레드 비동기 입출력 구조(메인-메뉴 출력 및 송신, 서브-서버 응답 실시간 수신)
-

개발 일정

6/2 (화)

- 기본적인 회로 구성
- TCP 서버/클라이언트 구현
 - 라즈베리파이에서 localhost로 서버-클라이언트 통신
- LED/부저/조도센서 기능 구현

- 공유 라이브러리
 - code/ 에 있는 src로 exec/lib/ 에 라이브러리 생성
 - server를 exec/lib/ 에 있는 라이브러리로 빌드
- 빌드 자동화 (Makefile 작성)

6/4 (목)

- 7세그먼트 기능 구현
- 우분투와 라즈베리파이 연결 후 테스트
- 서버 로그 작성 기능 추가 (server.log)
- 노래 선택 기능 추가

6/5 (금)

- 추가 기능 구현
- 동적 라이브러리
- 개발 문서, README.md 작성
- 회로도 작성

세부 구현 내용

① 하드웨어 제어 및 동적 라이브러리 모듈화 (.so)

- 각 주변 장치의 제어 코드(`led` , `buzzer` , `sensor` , `segment`)를 기능별 독립 폴더로 분리하여 관리
- 업그레이드 및 기능 수정 시 서버 전체를 재컴파일할 필요 없이 해당 `.so` 파일만 교체 (Dynamic Linking)하면 즉시 반영되도록 구현하여 아키텍처 유연성 확보
- `dlopen()` 을 이용해 서버에서 동적으로 링크

② 장치별 구현 기능

- LED
 - ON/OFF 및 소프트웨어 PWM 방식을 이용한 3단계 밝기(최대/중간/최저) 제어
- 부저

- 독립적인 재생 스레드를 통해 클라이언트 요청 시 음악 ON/OFF 제어
- **조도 센서**
 - 센서 모니터링 스레드를 백그라운드에서 구동하여 조도 값을 주기적으로 클라이언트에 전송
 - 빛이 감지되지 않으면(어두워지면) LED ON, 빛이 감지되면 LED OFF 자동 연동 제어 구현
- **7세그먼트**
 - BCD 디코더 인터페이스를 적용하여 클라이언트가 전송한 숫자(0~9)를 디스플레이에 표시
 - 독립 스레드를 생성하여 초 단위 카운트다운(-1씩 감소)을 수행하며, 0 도달 시 자동으로 부저 으뜸음(경보) 연동 제어
- **추가 기능**
 - 카운트다운 중에 SEGMENT STOP을 하면 카운트다운을 중지함
 - BUZZER ON을 하면 노래 3개 중 1개를 골라서 재생할 수 있음
 - 서버가 데몬 프로세스이기 때문에 서버 로그를 server.log에 출력

③ 멀티스레드 및 동기화 처리 (서버/클라이언트)

- **서버 자원 보호**
 - 7세그먼트 카운트다운 스레드와 메인 스레드 간의 `segment_state` 자원 접근 경쟁을 방지하기 위해 POSIX 뮤텝스 및 상호 배제(Mutex) 헬퍼 함수(`get/set`)를 도입하여 스레드 안전성 확보
- **클라이언트 비동기 구조**
 - 메뉴 입력 대기(`scanf`) 상태에서도 서버가 밀어내는 실시간 센서 데이터 및 카운트다운 완료 응답을 독립 수신 스레드(`receive_thread`)가 중단 없이 가로채 출력할 수 있도록 설계

④ 시스템 안정화 (데몬 및 시그널)

- **서버 데몬화**
 - `daemon(0,0)` API를 호출하여 세션 및 터미널 제어권 분리
 - 백그라운드 상시 서비스 형태로 구동 프로세스 최적화
- **시그널 방어벽**

- 클라이언트의 갑작스러운 연결 단선 시 서버가 동반 다운되는 것을 막기 위해 `SIGPIPE` 무시(`SIG_IGN`) 처리
- 터미널 오작동 방지를 위한 `SIGTSTP` 무시 처리 적용.

문제점 및 보완 사항

① 멀티스레드 비동기 출력에 따른 터미널 UI 비결정성 현상

- **현상:** 클라이언트 메인 스레드의 메뉴 출력 타이밍과 백그라운드 수신 스레드의 서버 응답 출력 타이밍이 동기화되지 않아, 어떤 명령(예: 부저 제어 등 스레드 조인 지연 발생 시)을 내리느냐에 따라 출력 순서가 매번 바뀌거나 메뉴판 하단이 흐트러지는 비결정적 UI 현상 발생.
- **원인:** 화면 출력 권한에 대해 두 스레드가 상호 배제나 차례를 맞추는 동기화 메커니즘이 부재하여 발생하는 멀티스레드 특유의 자원 경쟁 상태

보완 및 개선 방안

- **UI 렌더링 동기화 도입:** 수신 스레드가 서버 메시지를 출력한 직후 화면을 클리어(`system("clear")`)하거나 메인 UI 핸들러를 호출하여 메뉴를 강제로 재렌더링하는 구조로 보완 가능

② 문제점: 실시간 센서 데이터 출력으로 인한 CLI 메뉴 시인성 저해 (UX 결함)

- **현상:** 조도 센서를 켜면 서버로부터 1초마다 데이터(`[DATA] Sensor Value: X`)가 연속으로 수신되면서 기존 메인 메뉴 창이 터미널 상단으로 밀려나 화면에서 사라짐
- **원인:** 단일 화면 버퍼를 공유하는 표준 스트림 출력 방식을 사용하기 때문에, 실시간 데이터 출력이 사용자 메뉴 입력 인터페이스를 덮어쓰며 발생하는 레이아웃 충돌 현상임

보완 및 개선 방안

- **이스케이프 시퀀스 활용:** 리눅스 터미널 제어 커맨드(`\033[H\033[J` 등)를 사용하여 화면을 상단(메인 메뉴 고정)과 하단(실시간 데이터 흐름)으로 가상 분할하여 시인성 확보
- **`ncurses` 라이브러리 도입:** 리눅스 TUI(터미널 UI) 전용 라이브러리를 도입하여 메뉴 영역과 로그 출력 영역을 독립된 윈도우로 완벽히 분리·구현